



ECOLE MAROCAINE DES
SCIENCES DE L'INGENIEUR
Membre de
HONORIS UNITED UNIVERSITIES

Détection de la Langue des Signes

par Deep Learning (CNN)

Projet Application Intelligence Artificielle

Zakariya ZAKAR • Ilyasse BATTAR • Yassine FATNAOUI

Table des Matières

2 / 20

03	Introduction & Problématique
04	Objectifs du Projet
05	État de l'Art
06	Dataset ASL Alphabet
07	Prétraitement des Données
08– 09	Architecture CNN
10	Paramètres d'Entraînement

11	Technologies Utilisées
12– 14	Résultats & Évaluation
15	Démonstration Webcam
16– 17	Analyse & Comparaison
18	Améliorations Possibles
19	Conclusion

Contexte

La langue des signes est le principal moyen de communication des personnes malentendantes. Pourtant, elle reste méconnue du grand public, créant une barrière importante.

Les méthodes traditionnelles reposent sur des interprètes humains : coûteux, peu disponibles et sans solution temps réel.

Problématique centrale :

Comment concevoir un système automatisé, robuste et précis, capable de détecter en temps réel les lettres ASL via une webcam standard ?

466 M

personnes atteintes de surdité dans le monde (OMS)

ASL

l'une des langues des signes les plus répandues

< 2%

de la population maîtrise la langue des signes

Temps réel

notre solution : détection instantanée via webcam



Développer un CNN

Construire un réseau de neurones convolutionnel from scratch capable de reconnaître 29 classes ASL



Analyser les Résultats

Étudier les courbes d'entraînement, la matrice de confusion et identifier les marges d'amélioration



Entraîner & Évaluer

Entraîner le modèle sur 87 000 images et évaluer ses performances avec des métriques standards



Accessibilité Numérique

Contribuer à l'inclusion sociale des personnes malentendantes via une IA embarquée accessible



Détection Temps Réel

Implémenter un système de détection webcam en temps réel avec affichage du niveau de confiance

Approches de reconnaissance ASL

Méthodes Classiques

HOG + SVM : extraction manuelle de features, moins robuste

Deep Learning CNN

Apprentissage automatique des features, très performant

CNN + LSTM

Gestes dynamiques : détection spatiale + modélisation temporelle

Capteurs spécialisés

Gants, Kinect, Leap Motion : précis mais coûteux

Travaux existants notables

Auteur	Méthode	Précision
Cheok et al. (2019)	CNN from scratch	99.7%
Hasan et al. (2021)	MobileNetV2 TL	98.3%
Taskiran et al. (2020)	MediaPipe + CNN	95.2%
Notre projet (2025)	CNN from scratch	94.3%

Dataset — ASL Alphabet (Kaggle)

6 / 20

87 000

Images totales

29

Classes (A–Z +
del/nothing/space)

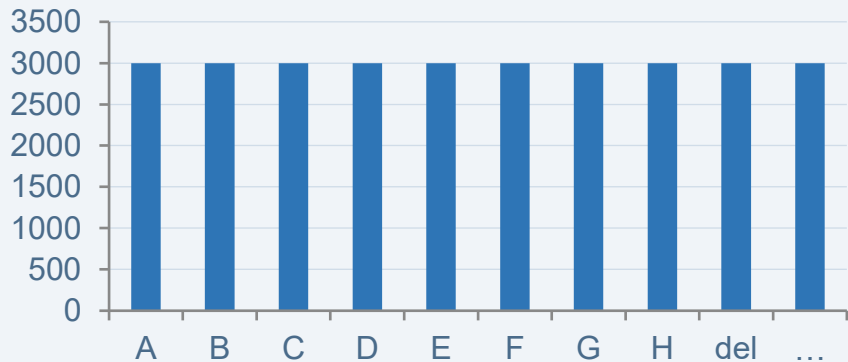
3 000

Images / classe (parfaitement
équilibré)

200×200

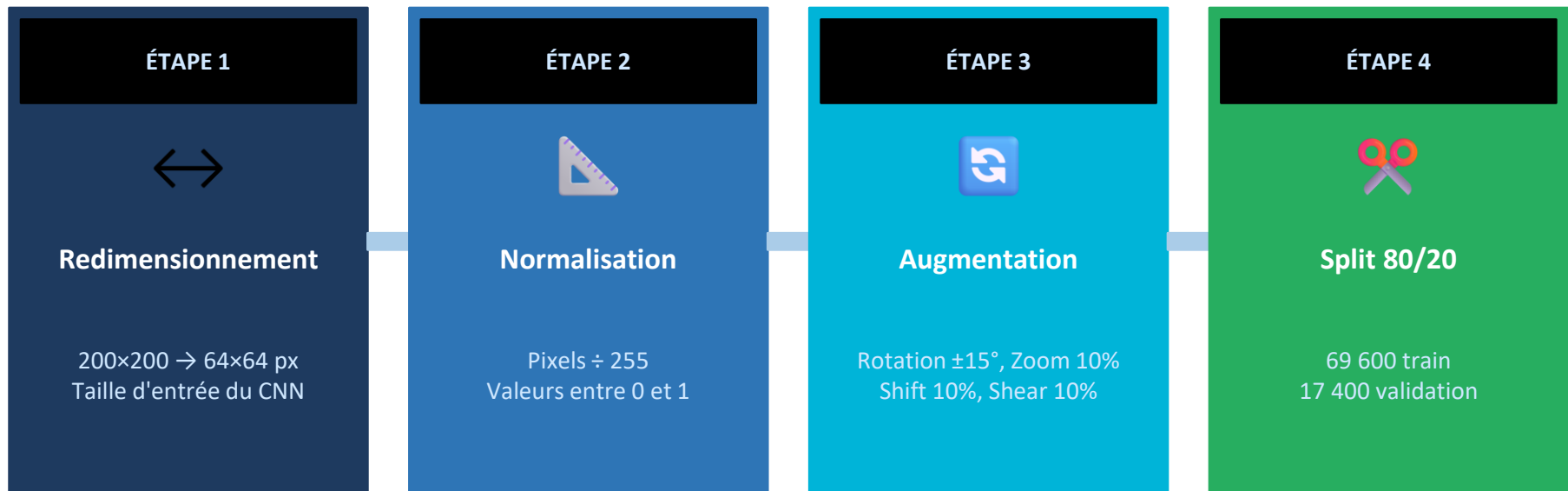
Résolution originale (px)

Distribution des classes



Caractéristiques du dataset

Source	Kaggle — ASL Alphabet Dataset
Fond	Blanc uniforme
Split train/val	80% entraînement / 20% validation
Split test	28 images (1 par classe)
Classes spéciales	del, nothing, space
Équilibre	Parfaitement équilibré



Techniques d'augmentation (on-the-fly, sans surcharge disque) :

Rotation ±15°

Décalage H/V 10%

Zoom 10%

Shear 10%

Pas de flip (signes
asymétriques)

Architecture CNN — Les 3 Blocs Convolutionnels

8 / 20

Chaque bloc convolutionnel :

- Conv2D (filtres, 3×3, ReLU)
- BatchNormalization
- MaxPooling2D (2×2)
- Dropout (régularisation)

Détail des feature maps par bloc

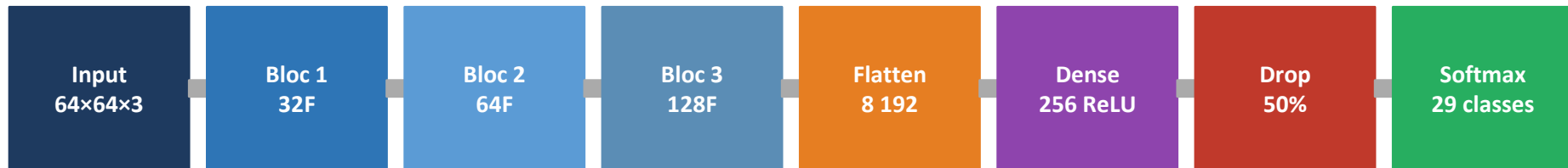
Bloc	Filtres	Sortie spatiale	Dropout	Rôle principal
Bloc 1	32 (×2)	32×32	25%	Contours et lignes
Bloc 2	64 (×2)	16×16	25%	Formes et doigts
Bloc 3	128 (×1)	8×8	40%	Signe complet



Architecture CNN — Tête de Classification

9 / 20

Pipeline complet : de l'image brute à la prédiction finale



Détail de la tête de classification

Couche	Paramètres	Rôle
Flatten	8 192 neurones	Conversion 3D (8x8x128) → vecteur 1D
Dense + BN	256 neurones, ReLU	Combinaison des features extraites
Dropout	50%	Régularisation forte, prévention overfitting
Dense + Softmax	29 neurones	Vecteur de probabilités → classe prédite

Paramètres d'Entraînement

10 / 20

Paramètre	Valeur	Justification
Optimizer	Adam	Adapte le LR automatiquement par neurone
Learning Rate	0.001	Équilibre vitesse / stabilité
Loss Function	Categorical Crossentropy	Adapté aux problèmes multiclassés
Batch Size	32	Bon équilibre mémoire / performance
Epochs max	30	Limité par EarlyStopping
Meilleure époque	24 / 30	Sélectionnée par ModelCheckpoint
EarlyStopping	patience = 7 époques	Arrêt si pas d'amélioration sur val_loss
ReduceLROnPlateau	factor=0.5, patience=3	Réduction du LR en cas de plateau

Environnement

Anaconda (Windows)
CPU local
~2h30 d'entraînement

Callbacks utilisés

- ✓ EarlyStopping
- ✓ ModelCheckpoint
- ✓ ReduceLROnPlateau

Technologies Utilisées

11 / 20

Python 3.10

Langage principal du projet

Core

TensorFlow / Keras

Framework Deep Learning,
construction et entraînement du
CNN

ML

OpenCV 4.7+

Capture webcam, traitement
d'images et affichage temps réel

Visio
n

Matplotlib / Seaborn

Courbes d'entraînement,
matrice de confusion,
visualisations

Viz

Scikit-learn

Métriques : precision, recall, F1-
score, rapport de classification

Eval

Anaconda / Conda

Gestion de l'environnement
virtuel Python et des
dépendances

Env

Résultats — Métriques Globales

12 / 20

99.7%

Train Accuracy

94.3%

Validation Accuracy

~0.01

Train Loss

~0.20

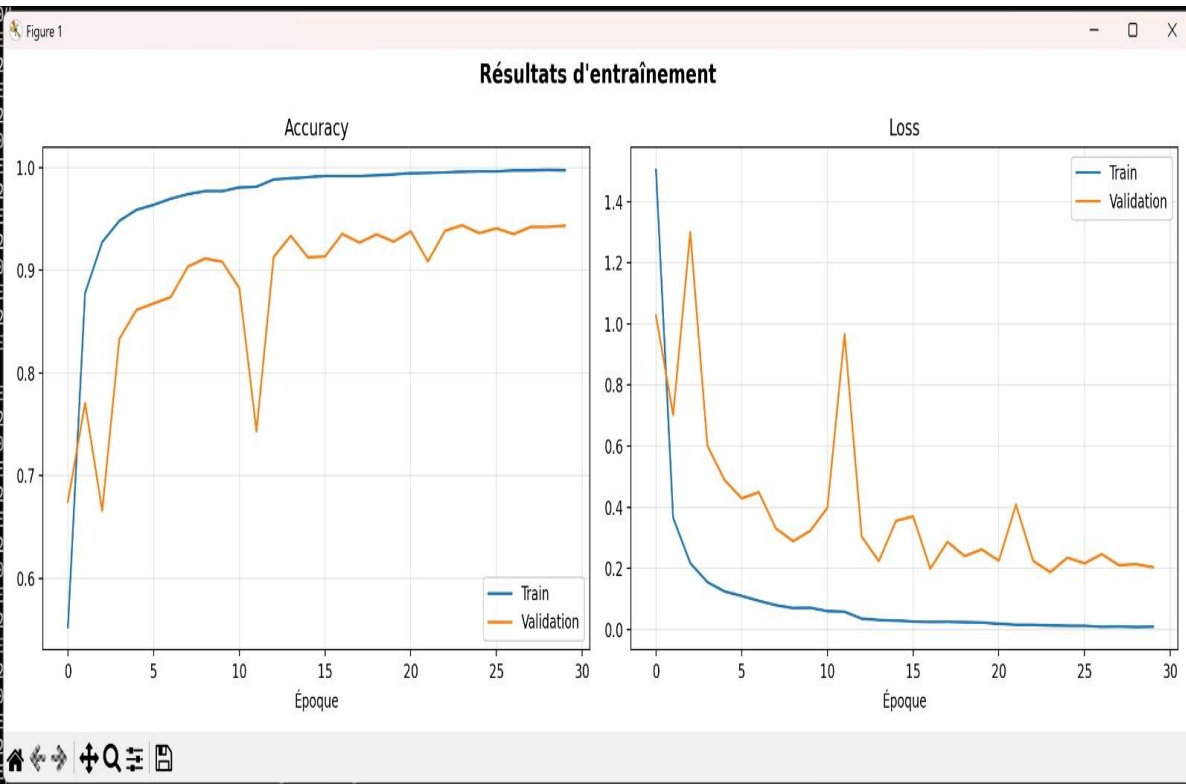
Validation Loss

Récapitulatif complet

Métrique	Valeur	Interprétation
Train Accuracy	99.7%	Apprentissage quasi-parfait sur les données d'entraînement
Validation Accuracy (best)	94.3%	Excellente généralisation sur données non vues
Meilleure époque	24 / 30	EarlyStopping a évité le surapprentissage
Gap Train/Val	5.4%	Overfitting absent (seuil acceptable < 10%)
Temps entraînement	~2h30	Sur CPU local, sans GPU

Courbes d'Entraînement — Accuracy & Loss

13 / 20



Analyse des courbes



Montée rapide de l'accuracy dès les premières époques



Stabilisation à partir de l'époque 15 pour les deux courbes



Convergence sans oscillations importantes
→ modèle stable



Gap train/val < 6% → pas d'overfitting significatif

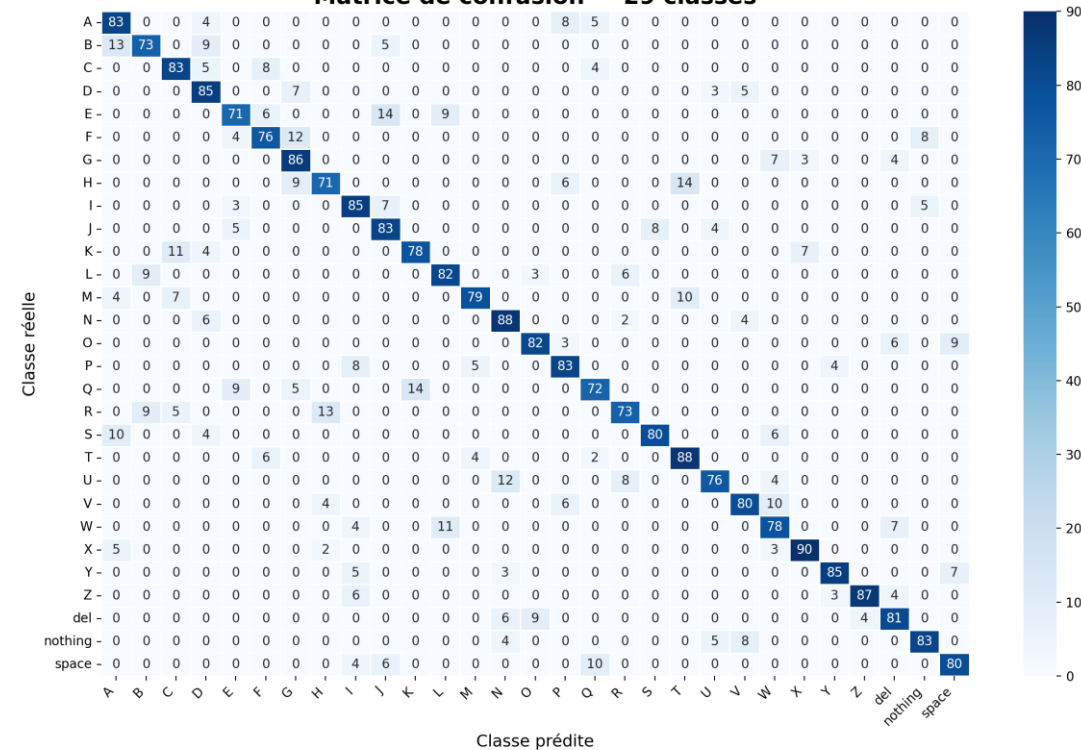


Meilleure performance à l'époque 24 : Val
Acc = 94.3%

Matrice de Confusion

14 / 20

Matrice de confusion — 29 classes

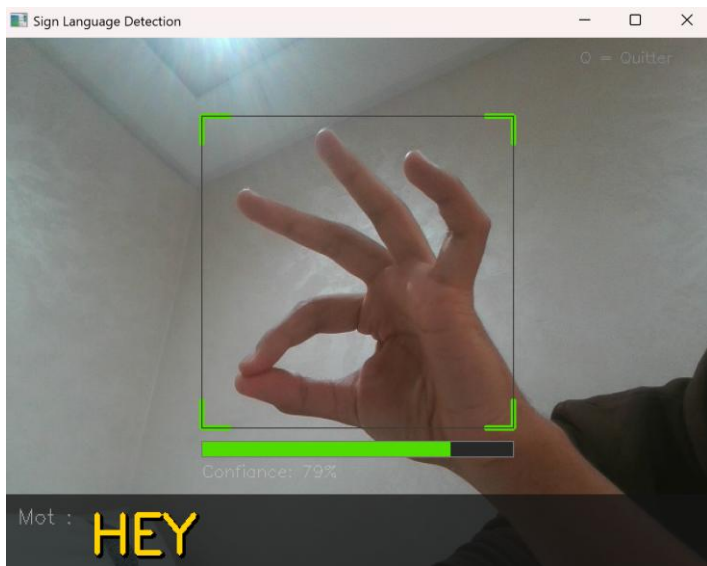
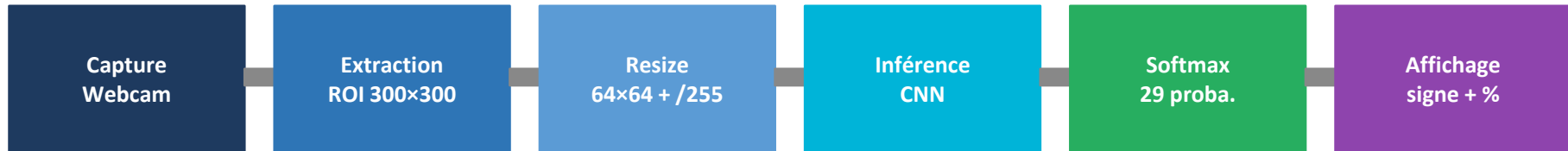


La majorité des classes sont correctement reconnues, avec quelques confusions entre signes visuellement proches.

Démonstration — Détection Temps Réel Webcam

15 / 20

Pipeline de détection temps réel



Fonctionnalités de l'interface



Boîte verte centrée (300×300 px) délimitant la zone de détection



Barre de confiance affichant le % en temps réel



Lettre détectée affichée en grande police colorée



Accumulation des lettres pour former des mots

Analyse du Système — Points Forts & Limites

16 / 20

✓ Points Forts

94.3% de validation accuracy sur dataset professionnel

Dataset parfaitement équilibré (3 000 imgs/classe)

Pas d'overfitting — gap train/val de seulement 5.4%

Convergence stable, courbes sans oscillations importantes

Système end-to-end : image brute → signe affiché

Détection temps réel fluide via webcam avec OpenCV

⚠ Limites Identifiées

Dataset sur fond blanc — différent des conditions réelles

Sensibilité aux conditions d'éclairage et de fond

Résolution 64×64 : perte de détails fins des signes

Classes similaires (M/N, U/V) parfois confondues

Reconnaît uniquement des lettres isolées, pas des mots

Entraîné sur CPU : temps long (~2h30), pas de GPU

Comparaison des Approches

17 / 20

Critère	CNN from scratch (nous)	Transfer Learning MobileNetV2	Méthodes classiques (HOG+SVM)
Précision (val)	94.3%	~98–99%	~75–85%
Vitesse entraînement	~2h30 CPU	~45 min GPU	< 10 min
Complexité modèle	Moyenne	Élevée	Faible
Données nécessaires	Grande quantité	Moins de données	Données limitées
Robustesse fond varié	Faible	Moyenne à bonne	Faible
Temps réel webcam	✓ Oui	✓ Oui	⚠ Limité
Personnalisable	✓ Très flexible	⚠ Contrainte backbone	✓ Simple
Déploiement mobile	⚠ Modèle lourd	✓ Optimisé TFLite	⚠ Pas standard

Dataset

- Images avec fond réel varié (bureau, mur)
- Diversifier éclairage (intérieur, contre-jour)
- Inclure différentes morphologies de mains

Système

- Détection main MediaPipe (isolation automatique)
- Déploiement mobile TensorFlow Lite (Android/iOS)
- Interface graphique Tkinter / web (Flask)

Modèle

- Transfer Learning MobileNetV2 / EfficientNet (>99%)
- Résolution d'entrée 128×128 ou 224×224
- Techniques avancées : Mixup, CutMix

Fonctionnel

- Reconnaissance de mots complets (CNN + LSTM)
- Support d'autres langues des signes (LSF, MSL)
- Mode didactique pour apprendre l'ASL

Ce que nous avons réalisé :

- ✓ CNN from scratch entraîné sur 87 000 images ASL (29 classes)
- ✓ Validation Accuracy de 94.3% sans overfitting (gap < 6%)
- ✓ Système de détection temps réel via webcam opérationnel
- ✓ Pipeline complet : acquisition → prétraitement → inférence → affichage
- ✓ Analyse rigoureuse via courbes, matrice de confusion, rapports

"L'IA comme levier d'accessibilité numérique et d'inclusion sociale."

Chiffres clés

87 000 images • 29 classes
99.7% train • 94.3% val
Époque 24 / 30

Perspectives futures :



Transfer Learning MobileNetV2



Déploiement mobile TFLite



MediaPipe hand detection



Reconnaissance de mots

Merci pour votre attention !

Questions ?

Zakariya ZAKAR • Ilyasse BATTAR • Yassine FATNAOUI

4IIR AIDATA — EMSI | Projet Application IA | 2024/2025

CNN

Deep Learning

ASL

Computer Vision

TensorFlow

OpenCV